Open-Source Audit – Python

STUDENT NAME	**ANSH ARORA**
REGISTRATION NUMBER	**24BAI10709**
SLOT	**B22**
DATE OF SUBMISSION	**26-03-2026**

1. Introduction

Open-source means that the source code of a software is available for anyone to look at, use, modify and share. This is different from software, where the code is private and controlled by a company. Open-source software is made in a way that's transparent and collaborative. Developers from all over the world can work together to improve the software fix problems and add new things. This makes source a way of thinking about sharing knowledge and working together not just a type of software.

Open source is really important today because it is used in everything we do with technology. From operating systems like Linux to programming languages like Python, open-source tools are the basis of software development. It also makes technology more accessible because people do not have to pay a lot of money to use tools. In my view open source also helps people learn because you can see how tasks are made of just using them without knowing how they work.

This project is about learning more about source by studying Python. Of just learning what Python can do the goal is to find out why it was made what problems it solves, what its license says and how it works with Linux. This project also includes writing shell scripts to show that I understand how open-source tools are used in systems. The goal of this project is to learn about Python but to understand the bigger idea behind open-source software and why it is so important, in the tech world. Overall open source and Python are the focus of this project and I want to learn as much as I can about open source and how it works with Python.

2. The Audit

Part A — Origin and Philosophy

A1. The problem this software was created to solve

Before Python came into the picture, programming was not very beginner-friendly. In my view programming languages like C and C++ were powerful. They were also quite complicated. Even writing programs with Python’s predecessors required a lot of syntax and attention to detail which could be frustrating especially for someone just starting out with language. It felt like you had to focus on how to write the code rather than what you actually wanted the software code to do.

Python was created by Guido van Rossum in 1991 in the Netherlands. His main idea for the language was pretty simple. He wanted to make a programming language that was easy to read and easy to use. Instead of making things more complex he focused

on simplicity and clarity for. He believed that programming with Python shouldn't feel like solving a puzzle every time you write Python code. What makes this different from languages is its clean and readable syntax. Unlike languages that use many symbols and complicated structures. It uses indentation and simple words, which makes code look almost like normal English. Because of this it becomes much easier to understand what's going on with Python if you didn't write the code yourself. Another important thing about this is that it was released as an open-source project. This means anyone could access and modify, contribute to it. Because of this a huge community of developers started improving Python over time. It wasn't controlled by a company, which allowed it to grow in many different directions based on real-world needs for Python. Today it is used everywhere. From web development to artificial intelligence and data science. In my experience with Python, I've noticed that it is much easier to pick up compared to languages like C++ or Java especially when starting as a beginner. In my view that's what Guido originally intended for the language. To make programming with this system more accessible and less intimidating.

Overall, it started as an attempt to simplify programming. Over time it has become one of the most important programming languages, in the world. The origin of Python clearly shows how focusing on simplicity and openness can lead to something that benefits a number of people accessing this coding language daily. It was made at a time when programming was getting more important. It was still hard for new people to learn. Guido van Rossum made Python when he was working at CWI in the Netherlands. He wanted Python to be better, than the ABC language. Python was meant to be used for applications not just for fun.

The person who made this coding language decided to share it with everyone so anyone could use it. Make it better. This was a part of why Python became so popular. Lots of people could work on this as their primary coding language. Make it what they wanted. That is why Python is still used by many people for so many different things today. It is used for tasks and Python is still a great language.

A2. The license — what it actually says

Python is released by the Python Software Foundation under a very open license. It's basically an open-source license that lets people do pretty much whatever they want with the code. You can use it, change it, and share it with others without having to worry about a long list of strict rules or legal headaches. The whole philosophy here is built on four simple ideas. You are free to use the language for any project, look at how it's built, and make your own changes to it. You're also allowed to share those

versions with everyone else. Because the licensing is so open, developers can dig into the code, tweak it to work better for them, and pass it along without getting into any trouble. Many people ask if a company can take the code, change it, and then sell their version without sharing their secrets. The answer is yes; they absolutely can. The Python Software Foundation License isn't strict at all, so businesses don't have to show their work to anyone else. This is a big reason why massive companies like Google, Netflix, and Instagram use it in their products—they get the power of the tool without the pressure to go public with their private code.

When you look at the GPL license versus the MIT license, the biggest difference is how much control they take. The GPL says if you change the software, you have to give those changes back to the community. The MIT license is much more laid back, just like Python's own license. It lets companies use the code in their own tools without being forced to share. If I were picking a license for a new project, I'd go with something like MIT or the PSF license because they just give people more freedom.

The four main things that free software gives you are: the ability to use the program for anything you want the ability to look at how the program works the ability to change it and the ability to share the original and the changed versions. Pythons license lets you do all these things, which makes Python an open and free software system.

A3. The ethics of open source

Open source is not just about sharing code but also about how we share knowledge and technology. A big question is, should all software be open source? On one hand if all software is source, it will be more transparent and more people can learn from it improve it and make new things. This can also reduce costs. Make technology available to everyone. However, on the hand some companies may not want to spend time and money building software if they have to give it away for free. Some proprietary software is needed for businesses to survive and compete.
Another issue is, is it right for companies to make money from open-source software? In my view this is a bit unclear. Companies like Google and Netflix use open-source technologies like Python and Linux to provide their services. They benefit from what others have done. They also improve these technologies and sometimes give back to the community. So, it is like a two-way relationship than taking advantage. If a company only takes from open source and does not give anything back it may not be fair. The phrase "standing on the shoulders of giants" means building ideas on what others have done. In source it means developers are not starting from nothing but are using and improving what others have created. In

my view open source helps this a lot as it allows innovation and working together. Of repeating what others have done developers can focus on solving new problems. It is also important to thank the creators and respect what they have done.

Overall In my view open source is very important in technology but it should exist with proprietary software. Both have their benefits and we need both. Open source helps people learn and work together while proprietary software helps drive business and innovation in a world.
In conclusion open-source software is really important for innovation, education and working together. It needs to work alongside proprietary software to keep things balanced between being open and making money. Open source helps people share knowledge and develop applications fast while proprietary software helps businesses run smoothly and have support. We need both to help the tech industry keep growing. Open source and proprietary software both have their strengths. A mix of both is good, for the tech industry.

Part B — Linux Footprint

This section shows how Python is structured and managed within a Linux system.

Installation Method

Python is available through the apt package manager in Linux. In my system, Python was already installed by default. It can also be installed using the command *sudo apt install python3*. This shows that Python is distributed as a package and does not require manual compilation.

```
anshere@LAPTOP-D96RU7G7:~$ python3 --version
Python 3.12.3
```

Figure1: python3 --version

Key Directories

The Python executable is located at */usr/bin/python3*, which I verified using the which python3 command. This shows how Linux organizes software into standard directories. Python, being a programming language, does not maintain a central system-wide configuration file in directories like */etc*, nor does it generate logs in */var/log* like typical server applications. However, tools built on Python such as pip store configuration files in hidden directories within the user's home folder.

```
anshere@LAPTOP-D96RU7G7:~$ which python3
/usr/bin/python3
```

Figure2: which python3

User and Group

Python runs under the current user. The permissions were checked using *ls -l /usr/bin/python3*, which shows the owner and access rights. This is important for system security. Python runs under the privileges of the current user. This is important for security because if a script is executed with root privileges, it can potentially access or modify critical system files. Running scripts as a normal user helps prevent unintended system-wide damage.

```
anshere@LAPTOP-D96RU7G7:~$ ls -l /usr/bin/python3
lrwxrwxrwx 1 root root 10 Aug  7  2024 /usr/bin/python3 -> python3.12
```

Figure3: ls -l /usr/bin/python3

Service Management

Python is not a system service. It runs only when a script is executed and does not run in the background. Therefore, commands like *systemctl* are not used.
Python is not something that you can manage like a system service on its own. You can manage services that are built using Python. For example, if you are building a web application using something like Flask or Django you can manage these services. You can start a Python service by using commands like
* *systemctl* start service-name
* *systemctl* status service-name
This is how Python works with Linux when you are using it for work. Python works with Linux service management when you are using Python for applications, like these.

Update Model

Python is updated using the apt package manager with commands *like sudo apt update* and *sudo apt upgrade*. This ensures updates from the open-source community.

```
anshere@LAPTOP-D96RU7G7:~/oss-audit-24BAI10709$ dpkg -l | grep python3
ii  libpython3-stdlib:amd64       3.12.3-0ubuntu2                         amd64        interact
ion)
ii  libpython3.12-minimal:amd64   3.12.3-1ubuntu0.3                       amd64        Minimal
ii  libpython3.12-stdlib:amd64    3.12.3-1ubuntu0.3                       amd64        Interact
```

Figure4: dpkg -l

Part C — The FOSS Ecosystem

Dependencies

Python needs help from open-source parts to work well. It uses system libraries that come with the operating system. It also needs compilers and runtime environments to function. For instance, it uses C libraries for speed. Tools like *pip* rely on network libraries and package repositories. This shows that open-source projects like Python are connected and not alone. They depend on each other to work. Python uses these dependencies to work. They help make it strong. The open-source projects are, like a team working together. They show that even Python, a language needs help to function.

Ecosystem and Impact

Because Python has so many useful tools and libraries, it is truly fantastic.
Python is used, for instance, to examine data using NumPy and Pandas. Additionally, they use Flask and Django to create websites. Some even use TensorFlow for machine learning. It is the foundation of all these tools. The primary benefit of Python is its utility. It is more than just a programming language; it's a flexible platform used in a variety of fields, such as web application development, automation, and artificial intelligence. Its widespread application in various fields is a direct consequence of its utility.

Role in Web / LAMP

Python is not usually part of the LAMP stack. The LAMP stack is made up of Linux, Apache, MySQL and PHP. Python does a similar job in modern web development. Frameworks like Django and Flask help developers build web applications using Python instead of PHP. Python works with databases like MySQL and PostgreSQL. It also runs on Linux servers like the traditional LAMP technologies. This shows how open-source ecosystems change over time. They replace parts with more flexible and scalable alternatives. Python serves this role.

The LAMP stack is no longer the option. It is an alternative. It is used by developers for web development. The use of Python, in web development is increasing. It has become a choice. Python and LAMP stack are used together. Python can also be used alone.

Community and Governance

The Python Software Foundation is the organization that manages the language. It oversees the development of Python and ensures that the project remains open-source for all. Developers around the world contribute to the improvement of the

software by posting their ideas on forums, mailing lists, or uploading their codes to GitHub. They also hold conferences where they discuss what the technology should be. When the Foundation decides to make changes, they adhere to a very straightforward procedure. Usually, this happens through Python Enhancement Proposals, or PEPs. Since many people are involved in making these decisions, the language stays useful and is constantly improving. The community really helps keep it relevant by ensuring the tools are aligned with the latest technological trends.

Part D — Open-Source vs Proprietary (Critical Analysis)

A comparison between Python and MATLAB is shown below:

DIMENSION	PYTHON (OPEN SOURCE)	MATLAB (PROPREITARY)
Cost	Free to use and distribute	Paid license required
Security	Source code is open and can be audited by anyone	Source code is closed and cannot be inspected
Support and Reliability	Community support, forums, and documentation	Official customer support provided by company
Freedom to Modify	Can modify and customize freely	Cannot modify core software
Community vs Corporate Control	Driven by global community (PSF)	Controlled by MathWorks company
Overall Verdict	Flexible and widely accessible	More controlled and specialized

After comparing Python and MATLAB it is clear that both have their advantages. It really depends on what you want to use them for. Python stands out because it is free. It is also very flexible. A lot of people support it. Python can be used for tasks. You can use it for web development. You can also use it for intelligence. On the hand MATLAB is different. It provides a controlled environment. It has dedicated support. This can be useful for engineering and research purposes.

As a student, I find Python more practical and accessible compared to MATLAB. Python is a choice for most applications. This is because it is easy to access. It is also very versatile.

Python allows developers to try things. They can modify things. They can build things without any restrictions .I would definitely recommend Python for use. I might even contribute to it in the future. However, MATLAB still has its place. It is useful in fields. These fields require tools. They also require support from MATLAB. MATLAB is still a choice, for those fields. Python and MATLAB both have their strengths.

3. Shell Scripts

Script 1- System Identity Report

Code:

```
(script1.sh)
#!/bin/bash

# Script 1: System Identity Report
# Author: Ansh Arora
# Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Ansh Arora"

# --- System Info ---
KERNEL=$(uname -r)
DISTRO=$(lsb_release -d | cut -f2)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date)

# --- Display ---
echo "======================================="
echo "   Open Source Audit - $STUDENT_NAME"
echo "======================================="

echo "Distribution : $DISTRO"
echo "Kernel       : $KERNEL"
echo "User         : $USER_NAME"
echo "Home Dir     : $HOME_DIR"
echo "Uptime       : $UPTIME"
echo "Date/Time    : $CURRENT_DATE"

echo "---------------------------------------"
```

```
echo "This system is typically licensed under"
echo "the GNU General Public License (GPL)."
echo "--------------------------------------"
```

```
anshere@LAPTOP-D96RU7G7:~/oss-audit-24BAI10709$ ./script1.sh
========================================
   Open Source Audit - Ansh Arora
========================================
Distribution : Ubuntu 24.04.1 LTS
Kernel       : 6.6.87.2-microsoft-standard-WSL2
User         : anshere
Home Dir     : /home/anshere
Uptime       : up 4 hours, 6 minutes
Date/Time    : Wed Mar 25 22:51:05 UTC 2026
----------------------------------------
This system is typically licensed under
the GNU General Public License (GPL).
----------------------------------------
```

Output- system identity report

Explanation:

This script displays basic information about the Linux system in a formatted way. It shows details like the distribution name, kernel version, current user, home directory, uptime, and date/time. It uses variables, command substitution, and echo statements to present the information clearly.

Script 2 — FOSS Package Inspector

Code:

```
#!/bin/bash

# Script 2: FOSS Package Inspector
# Author: Ansh Arora

# Chosen package
PACKAGE="python3"

# Check if package is installed
if dpkg -l | grep -q "$PACKAGE"; then
    echo "$PACKAGE is installed."

    # Display package details (version, description)
    dpkg -l | grep "$PACKAGE"

else
```

```
    echo "$PACKAGE is NOT installed."
fi

# Description using case statement
case $PACKAGE in
    python3)
        echo "Python: A powerful open-source programming
language." ;;
    git)
        echo "Git: A distributed version control system." ;;
    vim)
        echo "Vim: A highly configurable text editor." ;;
    *)
        echo "No description available." ;;
esac
```

```
anshere@LAPTOP-D96RU7G7:~/oss-audit-24BAI10709$ ./script2.sh
python3 is installed.
ii  libpython3-stdlib:amd64      3.12.3-0ubuntu2                    amd64        interactive high-level object-ori
ion)
ii  libpython3.12-minimal:amd64   3.12.3-1ubuntu0.3                 amd64        Minimal subset of the Python lang
ii  libpython3.12-stdlib:amd64    3.12.3-1ubuntu0.3                 amd64        Interactive high-level object-ori
rsion 3.12)
ii  libpython3.12t64:amd64        3.12.3-1ubuntu0.3                 amd64        Shared Python runtime library (ve
ii  python3                       3.12.3-0ubuntu2                   amd64        interactive high-level object-ori
ion)
```

```
ii  python3-yaml                  6.0.1-2build2                     amd64        YAML parser and emitter for
ii  python3-zope.interface        6.1-1build1                       amd64        Interfaces for Python3
ii  python3.12                    3.12.3-1ubuntu0.3                 amd64        Interactive high-level objec
ii  python3.12-minimal            3.12.3-1ubuntu0.3                 amd64        Minimal subset of the Python
Python: A powerful open-source programming language.
```

Output- FOSS Package Inspector

Explanation:

This script checks whether a specific package (python3) is installed on the system using the dpkg command. If installed, it displays package details, and it also uses a case statement to print a short description of the software. It demonstrates conditional statements and pattern matching.

Script 3 — Disk and Permission Auditor

Code:

```
#!/bin/bash

# Script 3: Disk and Permission Auditor
# Author: Ansh Arora

# List of important directories
```

```
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

# Loop through each directory and collect details
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then

        # Get permissions, owner, group
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')

        # Get directory size
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        echo "$DIR -> Permissions: $PERMS | Size: $SIZE"

    else
        echo "$DIR does not exist on this system"
    fi
done
```

```
anshere@LAPTOP-D96RU7G7:~/oss-audit-24BAI10709$ ./script3.sh
Directory Audit Report
----------------------
/etc -> Permissions: drwxr-xr-x root root | Size: 4.2M
/var/log -> Permissions: drwxrwxr-x root syslog | Size: 270M
/home -> Permissions: drwxr-xr-x root root | Size: 456K
/usr/bin -> Permissions: drwxr-xr-x root root | Size: 135M
/tmp -> Permissions: drwxrwxrwt root root | Size: 28K
```

Output- Disk and Permission Auditor

Explanation:

This script audits important system directories by looping through them and displaying their permissions, ownership, and disk usage. It uses a for loop along with commands like ls, awk, and du to extract and display the required information.

Script 4- Log File Analyzer

Code:

```
#!/bin/bash

# Script 4: Log File Analyzer
# Author: Ansh Arora
# Usage: ./script4.sh /var/log/syslog

LOGFILE=$1
```

```
KEYWORD=${2:-error}   # Default keyword is 'error'
COUNT=0

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Read file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

# Output
# Show last 5 matching lines
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
```

```
anshere@LAPTOP-D96RU7G7:~/oss-audit-24BAI10709$ ./script4.sh /var/log/syslog
Keyword 'error' found 582 times in /var/log/syslog
```

Output- Log File Analyzer

Explanation:

This script analyses a system log file by reading it line by line and counting how many times a specific keyword (such as "error") appears. It uses a while loop, conditional statements, and a counter variable. It also accepts command-line arguments, making it flexible to analyse different log files and keywords.

Script 5- The Open Source Manifesto Generator

Code:

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator
# Author: Ansh Arora

echo "Answer three questions to generate your manifesto:"
echo ""

# User input
read -p "1. Name one open-source tool you use: " TOOL
read -p "2. What does freedom mean to you? " FREEDOM
read -p "3. What would you like to build? " BUILD
```

```
# This script uses user input, string concatenation, and file
redirection

# Date and output file
DATE=$(date "+%Y-%m-%d")
OUTPUT="output_$(whoami).txt"

# Create paragraph
echo "----- Open Source Manifesto -----" > $OUTPUT
echo "On $DATE, I believe that open source tools like $TOOL
represent true innovation. Freedom to me means $FREEDOM. Using
this freedom, I aspire to build >

# Display output
echo "Manifest saved in $OUTPUT"
cat $OUTPUT
```

```
anshere@LAPTOP-D96RU7G7:~/oss-audit-24BAI10709$ ./script5.sh
Answer three questions to generate your manifesto:

1. Name one open-source tool you use: python
2. What does freedom mean to you? freedom
3. What would you like to build? app
Manifest saved in output_anshere.txt
----- Open Source Manifesto -----
On 2026-03-25, I believe that open source tools like python repr
p and contribute back to the community.
```

Output- The Open Source Manifesto Generator

Explanation:

This script generates a personalized open-source manifesto by taking user input through the terminal. It combines the inputs into a meaningful paragraph using string concatenation and saves it to a text file. It demonstrates user input handling, file writing, and the use of the date command.

4. Conclusion

This project really helped me understand open-source software and how important it is for computing. I learned that Python is an open-source language that's easy to use and can be used by many people. The project also taught me about Linux and how it works inside a computer system. I compared open-source software to software and saw that it is crucial to pick the right tools for the job.

The open-source software project was an experience. It helped me learn more about open-source technologies and scripting at the system level. I got to know open-source software and how it works. Open-source software is very important for computing.

I used Python for this project. It was really helpful. Python is an open-source language. It is very accessible. The Linux analysis was also very useful. It showed me how software works in a system.

The project was a success. I learned a lot from it. I learned about open-source software and its importance. I also learned about the difference, between source and proprietary software. Open-source software is the way to go for people.